Set Data Structures

Part 6

1

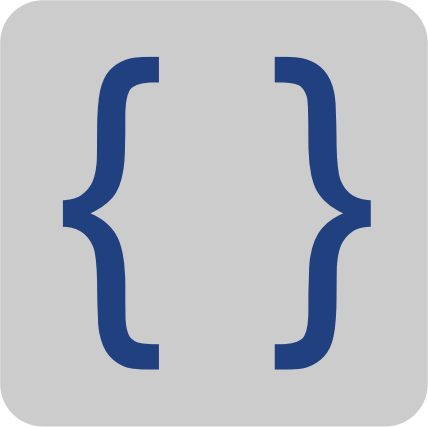
Importance of Sets

Organizing Information

2

Importance of Sets

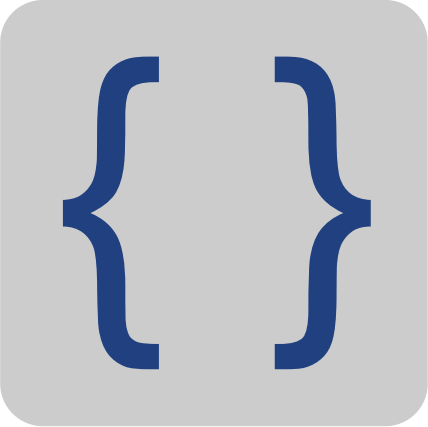
- A *set* is an unordered collection of "objects"
- Sets are used in computer science is a wide variety of ways



3

Importance of Sets

- Depending on the attributes of the set, and how we use it, there are different approaches
- Programmers must choose the best model given how it will be used



4

Set Review: Membership

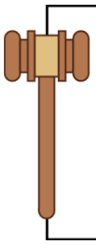
- Set notation uses a special symbols to denote if an object is a member of a set
- Below, the set V contains vegetables

```
potato ∈ V
bacon ∉ V
```


5

Set Review: Union

- A union of two sets combines all members of each set into a new one
- So, the result is two merged sets

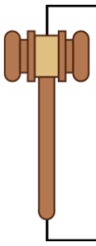


$$A \cup B = \{ x \mid x \in A \text{ or } x \in B \}$$

6

Set Review: Intersection

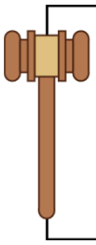
- The intersection of two sets contains only those elements that are found in both sets
- So, the result is where the two sets overlap



$$A \cap B = \{ x \mid x \in A \text{ and } x \in B \}$$

Set Review: Difference

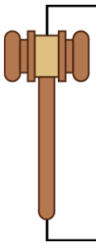
- *Difference* (aka exclusion) removes all items found in set from another
- Typically, it is written either $A - B$ or $A \setminus B$



$$A - B = \{ x \mid x \in A \text{ and } x \notin B \}$$

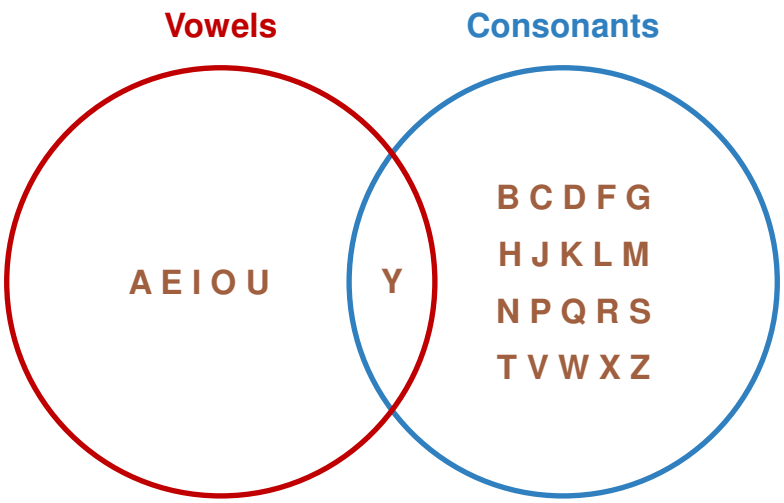
Set Review: Complement

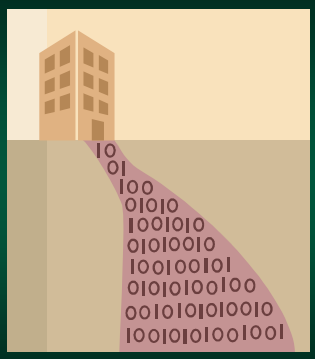
- The *complement* of a set A, is all elements in the Universe, <u>not</u> in A
- Typically written as A' or A^c
- Alternatively written using an overbar



$$A' = \{ x \mid x \notin A \}$$

Example: Letters in English





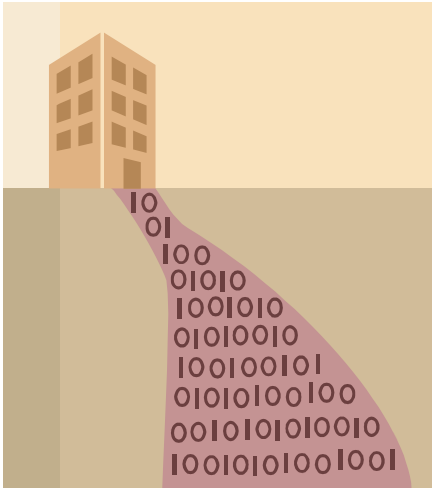
Bit Vectors

Sets and Bits

11

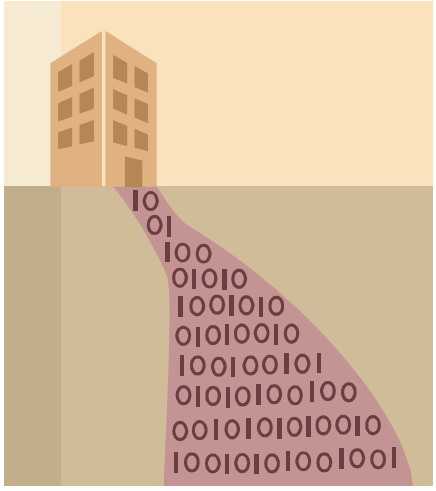
Bit Vectors

- A *bit vector* can store finite, countable sets using bits
- Also known as a *bit array*, *bit set*, and *bit map*
- Compact format that can perform a set operations with a single operation *(fast!)*



12

Bit Vectors



- Each object in the universe is represented as a <u>single</u> bit in the string of bits
- If $x \in A$, then the bit is **1**, otherwise **0**

Example 1

- U = (deadpool, hulk, gambit, wolverine, torch)
- A = { deadpool, wolverine }

```
U = 11111
A = 10010
```

Example 1

- U = (fry, leela, zoidberg, bender, hermes)
- A = { fry, leela, bender }

```
U = 11111
A = 11010
```


15

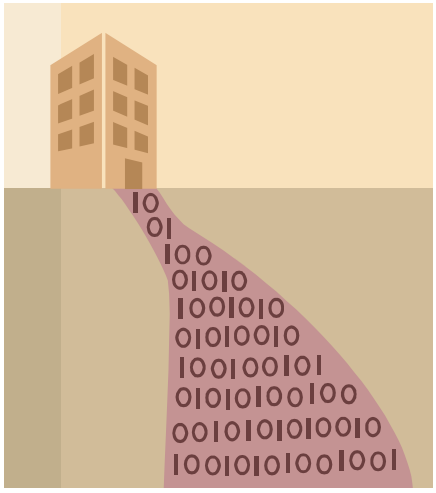
Example 3

- U = (2, 3, 5, 7, 11, 13, 17, 19)
- A = { 3, 5, 11, 19 }

```
U = 11111111
A = 01101001
```


16

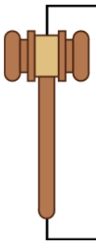
Why this is useful



- Computers can easily perform and & or operations on bytes (or multiple bytes)
- This means set operations can be performed amazingly fast

Let's look at the definitions again…

- The definitions of union and intersection are nearly identical
- The relationship between the elements is defined using an *AND* or *OR*



$$A \cup B = \{ x \mid x \in A \text{ or } x \in B \}$$
$$A \cap B = \{ x \mid x \in A \text{ and } x \in B \}$$

Let's look at the definitions again…

- We can apply *bit-wise-and* & *bit-wise-or*
- It will apply the operation to each of the bits in matching columns

$$A \cup B = \{ x \mid x \in A \text{ or } x \in B \}$$
$$A \cap B = \{ x \mid x \in A \text{ and } x \in B \}$$

Let's look at the definitions again…

- So, each bit in A will be compared to its matching bit in B
- Bit match can do sets!

$$A \cup B = \{ x \mid x \in A \text{ or } x \in B \}$$
$$A \cap B = \{ x \mid x \in A \text{ and } x \in B \}$$

Example: Union (using or)

```
U = {a, b, c, d, e, f, g}
A = {b, c, d} = 0111000
B = {d, e, f} = 0001110

    0111000
 or 0001110
    0111110 = {b, c, d, e, f}
```


21

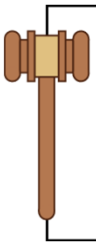
Example: Intersection (using and)

```
U = {a, b, c, d, e, f, g}
A = {b, c, d} = 0111000
B = {d, e, f} = 0001110

    0111000
and 0001110
    0001000 = {d}
```


22

Complement

- Then, how do we do a complement of a set A?
- We must flip all the bits from 1 to 0, and 0 to 1
- We can use a *binary-not* or the *XOR* operation



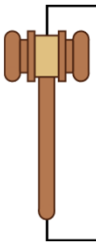
$$A' = \{ x \mid x \notin A \}$$

Example: Complement (using not)

```
U = {a, b, c, d, e, f, g}
A = {b, c, d} = 0111000

not 0111000
    1000111 = {a, e, f, g}
```

Exclusion

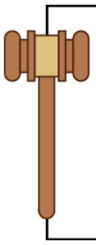
- Finally, how do we do set difference?
- The "subtract" operator will <u>not</u> work
- Let's look at the definition a bit more closely



$$A - B = \{ x \mid x \in A \text{ and } x \notin B \}$$

Exclusion

- It's essentially the definition of *intersection*
- Except, the second operand is the definition of *complement*.



$$A - B = \{ x \mid x \in A \text{ and } x \notin B \}$$

Example: Intersection (using and)

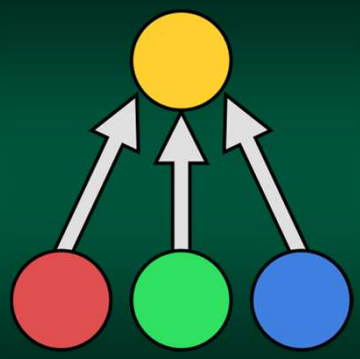
```
U = {a, b, c, d, e, f, g}
A  = {b, c, d, f} = 0111010
B  = {d, e, f}    = 0001110
B' = not 0001110  = 1110001

    0111010
and 1110001
    0110000 = {b, c}
```


Java/C Code

```
Intersection :  a & b
Union        :  a | b
Complement   :  ~a
Exclusion    :  a & ~b
```

&& and || are Boolean operators. These are bit-wise.

The tilde ~ is a bitwise not.



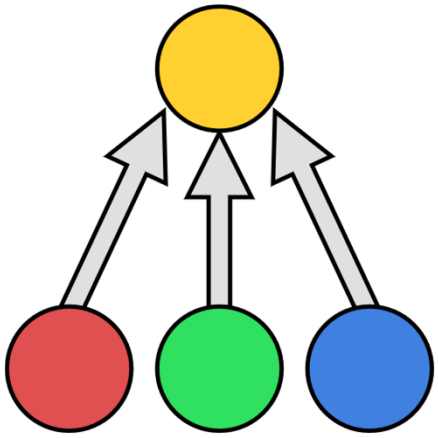
Union-Find

When it absolutely, positively has to be unioned overnight … er… O(1)

29

Union-Find

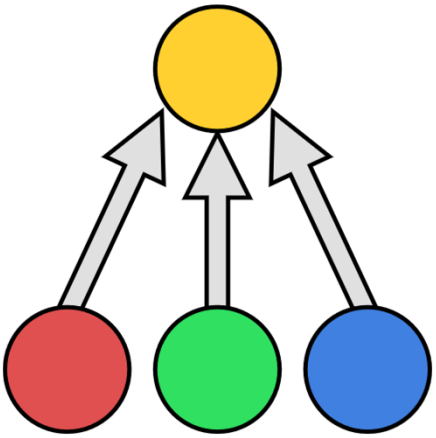
- Often there are situations where we have a number of known objects
- … i.e. a finite countable set
- … and we want to quickly union them together



30

Union-Find

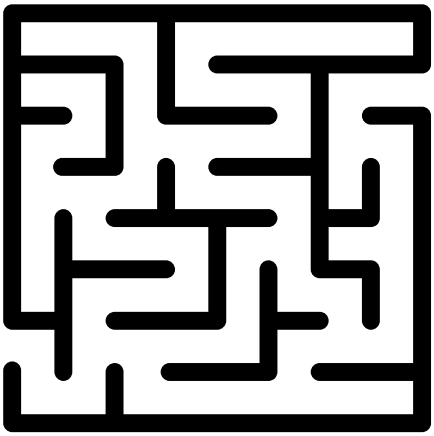
- *Union-Find* data structure maintains a list of nodes *partitioned* into *disjoint* subsets
- e.g. { {a} {b} {c} {d} {e} } is a full partition of {a, b, c, d, e}



31

Example Applications

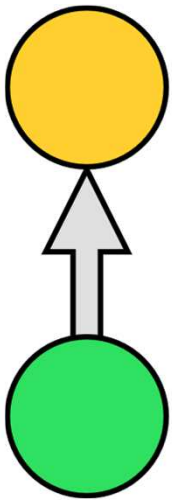
- Kruskal's Tree-spanning Algorithm – uses a set of known edges
- Maze creation algorithms use it to track sets of connected paths – so the maze is always solvable



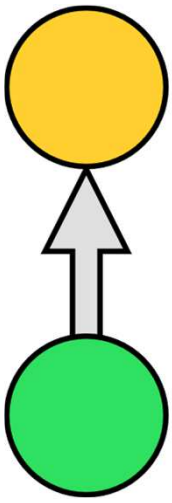
32

The Approach

- Sets are stored as a variation of the classic tree
- However, in this approach children link to their parents
- So, the branches point *"backwards"* from standard trees – i.e. upwards

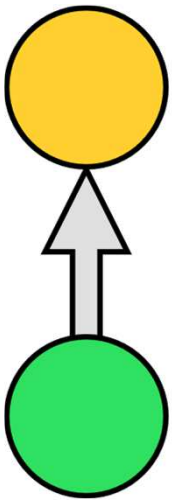


Arrangement of Nodes



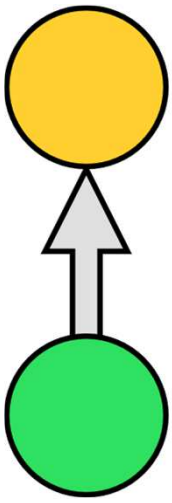
- A parent node can have multiple children - this is **not** a binary tree!
- Every node - in the tree - is part of the same set

Arrangement of Nodes



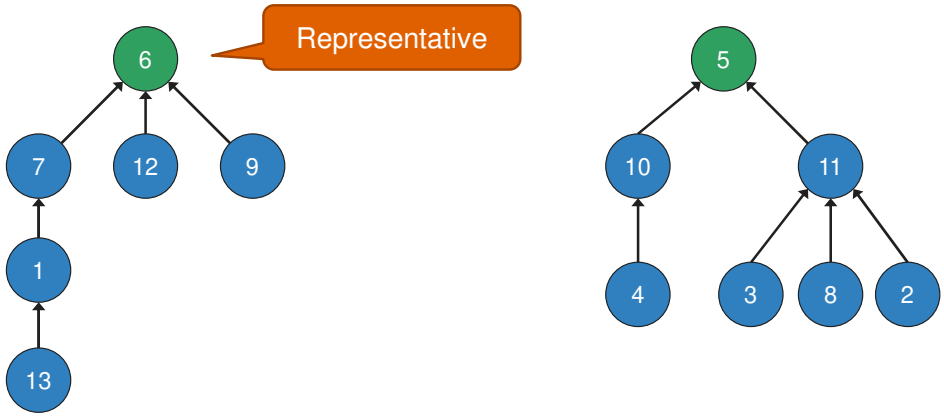
- The root is called the *representative* of the set
- This node is **not** special nor is its value special
- … it just happens to be the node the represents the set

Arrangement of Nodes



- Any node can "find" its representative – by following the upwards branches
- If any two nodes have the same representative, then they are in the same set

Examples

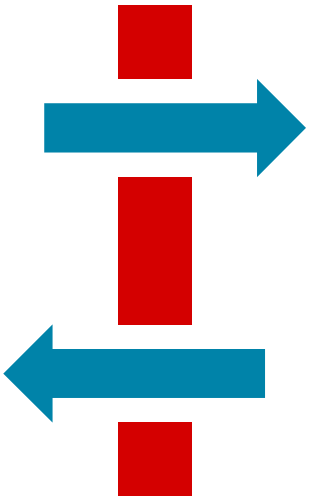


{ 1, 6 ,7, 9, 12, 13 }

{ 2, 3, 4, 5, 8, 10, 11 }

Union-Find Operations

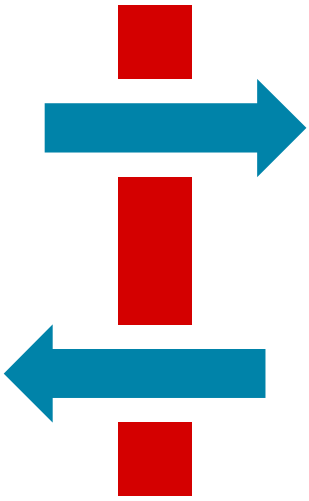
- The Union-Find data structure contains 3 operations that handle sets
- All 3 can modify the structure of the tree – which automatically optimizes itself into **O(1)**



Union-Find Interface

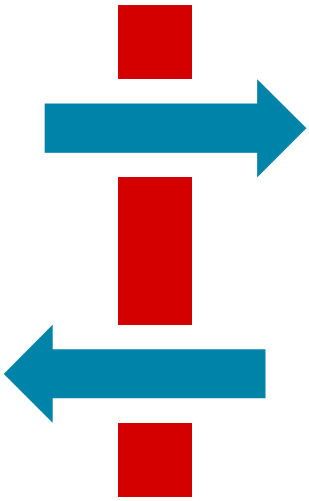
public class UnionFind	
void	makeSet(object value)
void	union(object a, object b)
object	find(object a)

Makeset



- Creates a set (within the Union-Find instance) that contains only a single element
- Also known as a *singleton*
- Essentially, this is the same as an “add” for the ADT

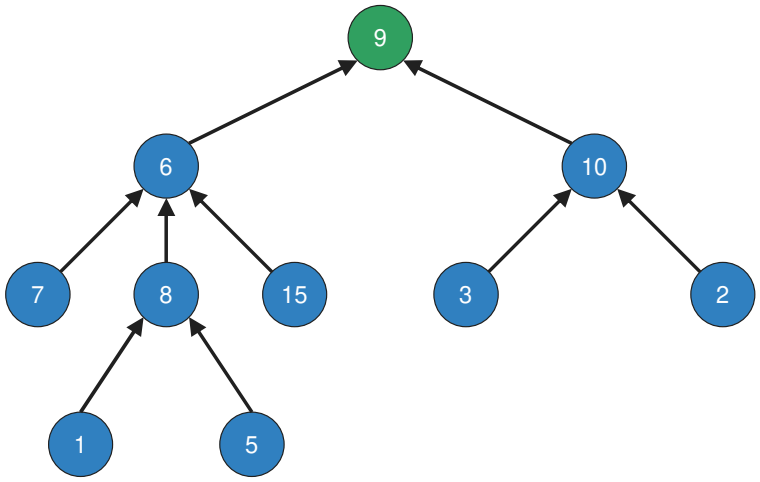
Find



- The *find operation* starts with a node and locates its *representative*
- Travels up the tree until it finds a node without a link (which is the representative)

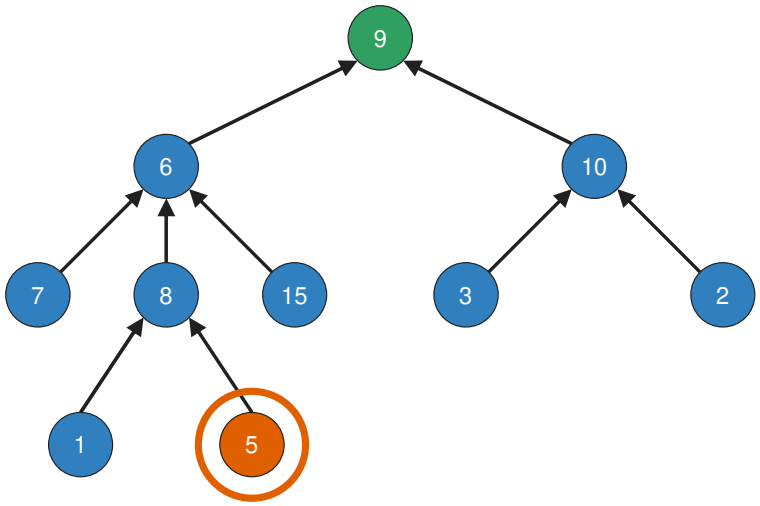
41

Find(5) → 9



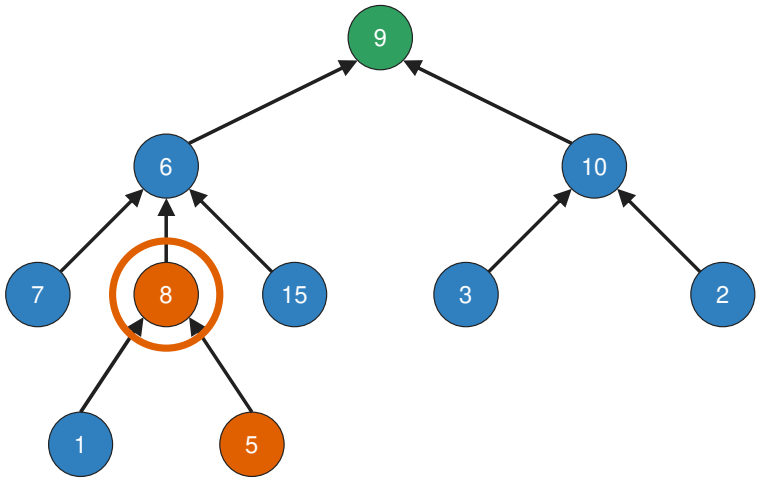
42

Find(5) → 9



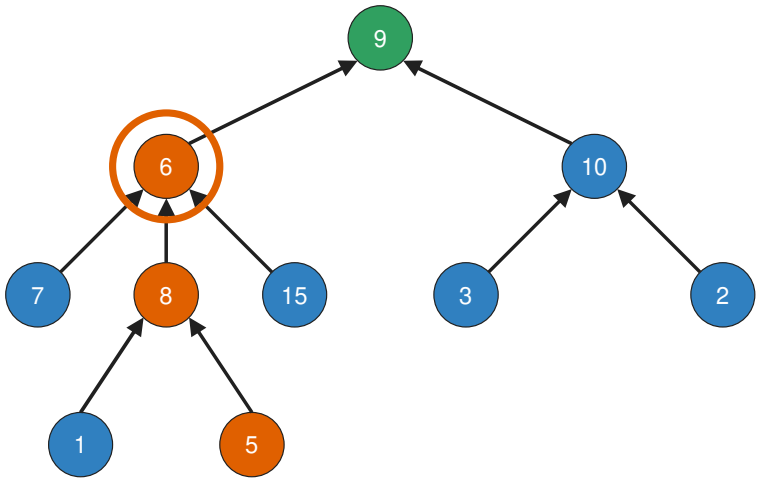
43

Find(5) → 9



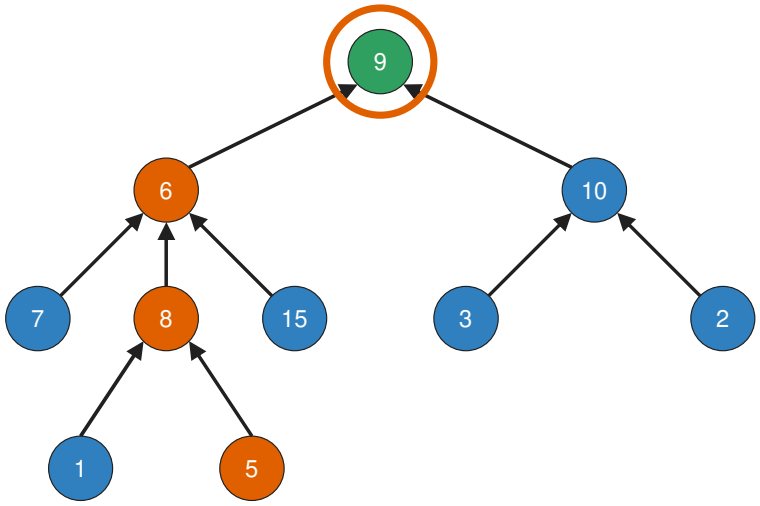
44

Find(5) → 9



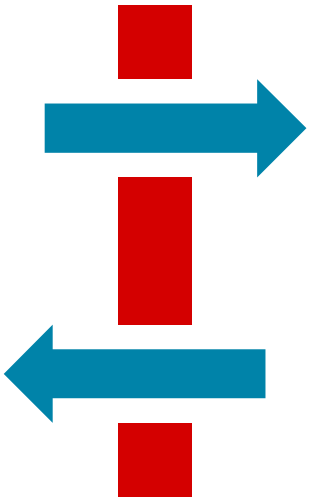
45

Find(5) → 9



46

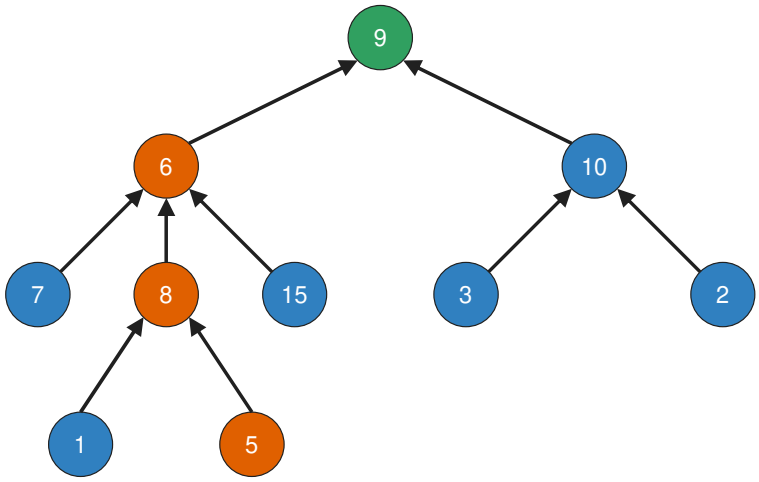
Find Automatically Optimizes the Tree



- After the answer is found, all nodes, along the path, are pointed to the representative
- This means the tree optimizes by becoming "flattened" (and faster) with each find

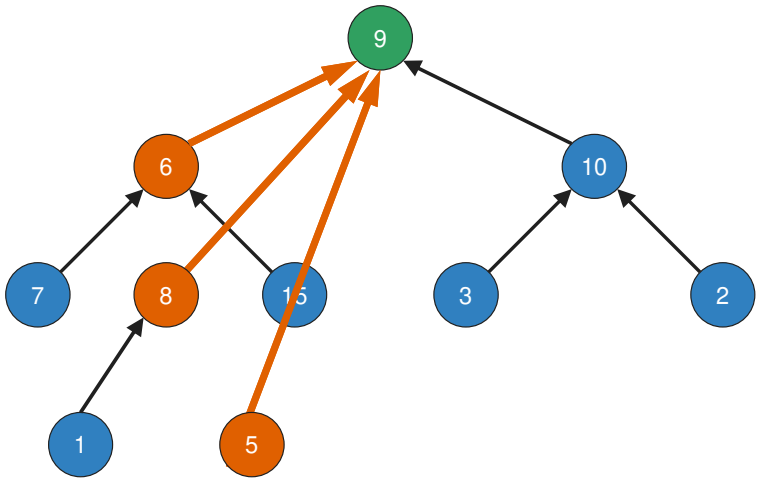
47

Connect Each Node to Representative

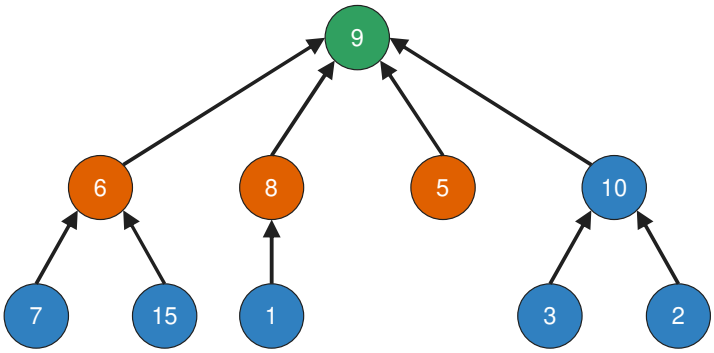


48

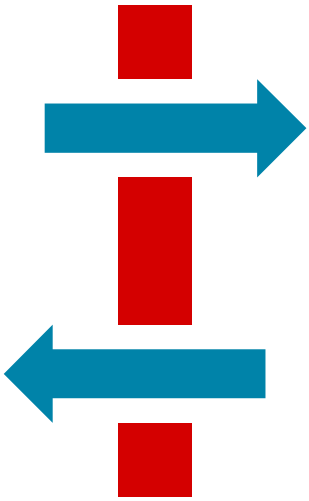
Connect Each Node to Representative



Updated, Flattened, Tree



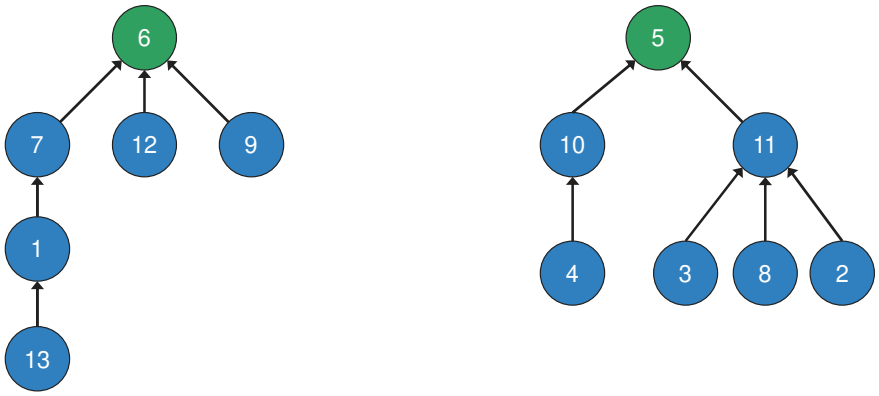
Union



- Combines two sets into a single set
- This is accomplished by updating a single parent link
- So, one representative will be pointed at another representative

So, which one do we choose?

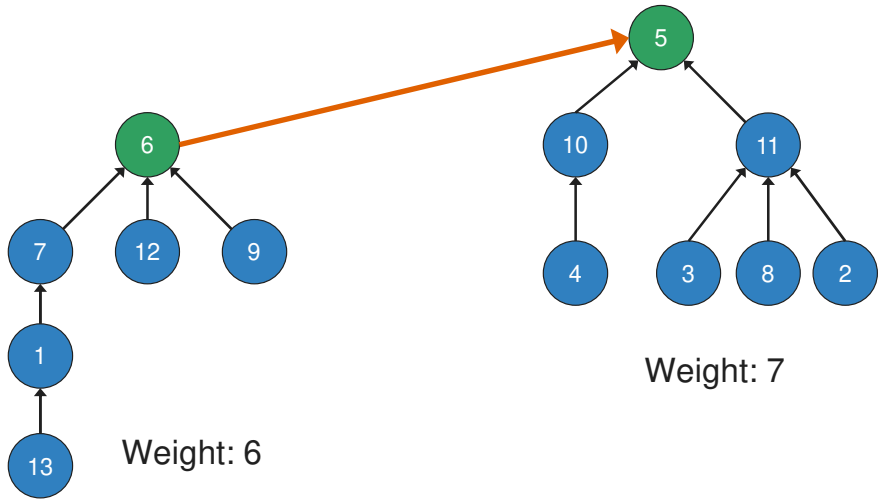
Which will be the new representative?



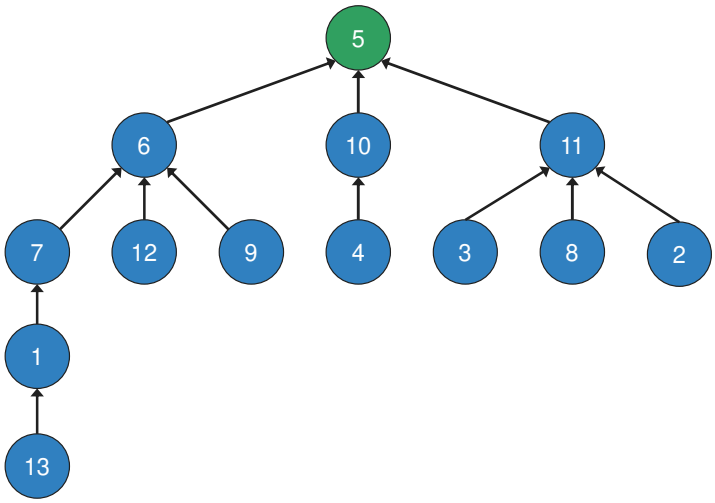
Approach #1: Union by Weight

- Using *Union by Weight*, the tree with the fewer nodes (not edge weight) is made a subtree of the larger tree
- This helps create a more balanced union in terms of weight
- Again, the Find operation will optimize the tree

So, which one do we choose?



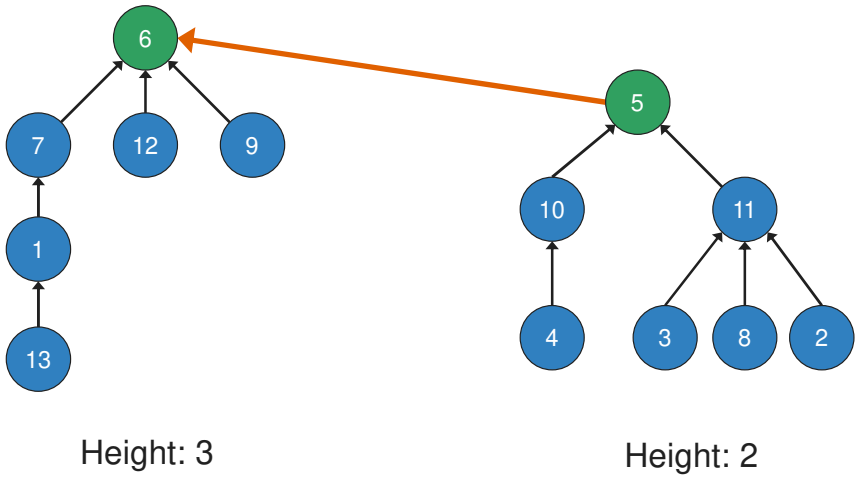
Approach #1: Result



Approach #2: Union By Height

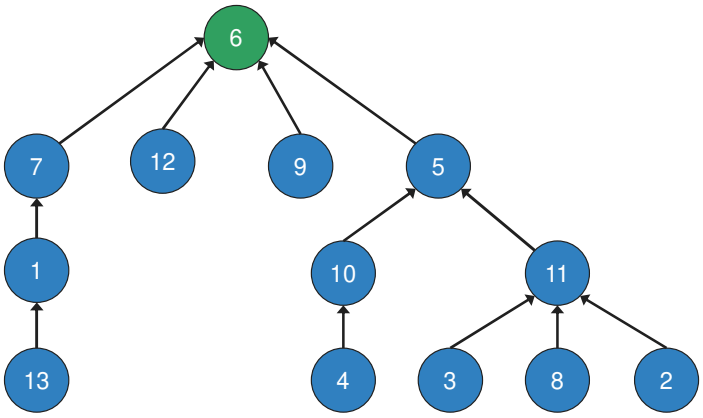
- Using *Union by Height*, the tree with the smaller height is made a subtree of the taller tree
- This helps create a more balanced union in terms of height
- But, Find, will fix this automatically

Approach #2: Union By Height

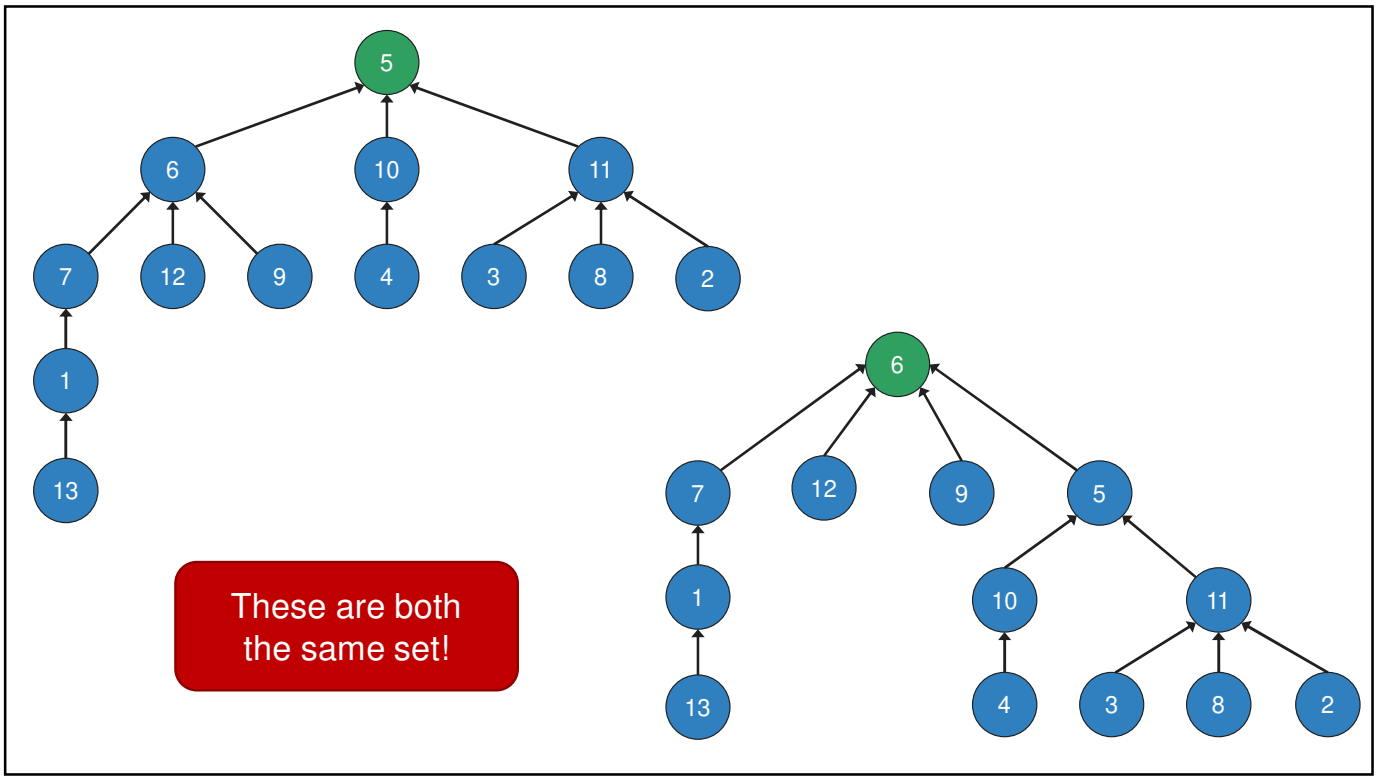


57

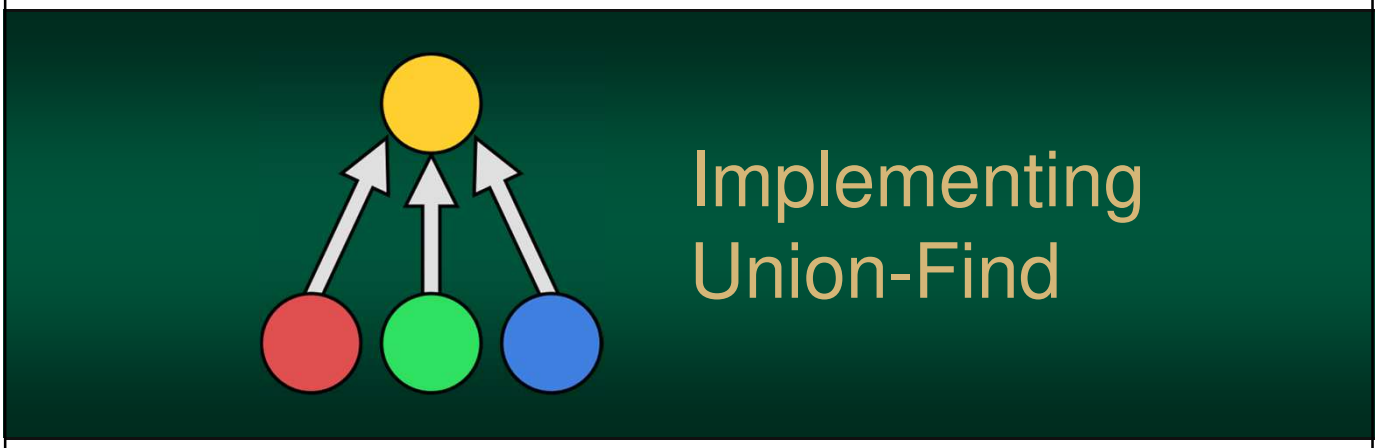
Approach #2: Result



58



59



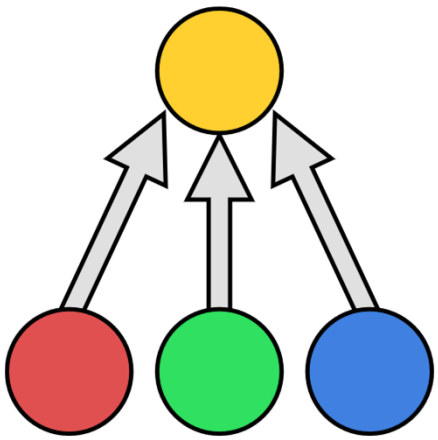
Implementing Union-Find

We have two options

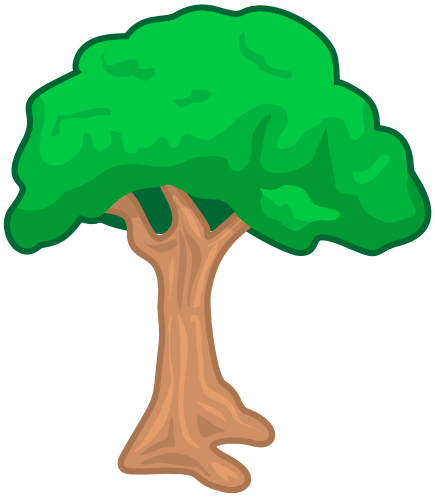
60

Implementing Union-Find

- Like the Stack and Queue, a Union-Find can be implemented using different data structures
- In this case, we have two good options: arrays and trees



Implementing with an Tree



- A tree solution is pretty straight forward
- The node, in this case, has a link to the parent node
- Recursion will occur within the node class itself

Union-Find Node

```
class Node
    public Object value;      //Anything
    public Node parent;
end class
```


63

Find Node Function

```
function find() returns Node
    if this.parent is null
        answer = this              ...This is the representative
    else
        answer = this.parent.find()          ...Recursion
        this.parent = answer                 ...Relink
    end if

    return answer
end function
```


64

Implementing with an Array

- A Union-Find can also be implemented using an array
- Recursion will be based on the index of an array element
- Notice the code is very similar to the tree solution.

65

Find Array Function

```
function find(int index) returns int
    if table[index].parent = index
        answer = index            ...This is the representative
    else
        answer = find(table[index].parent)       ...Recursion
        table[index].parent = answer             ...Relink
    end if

    return answer
end function
```

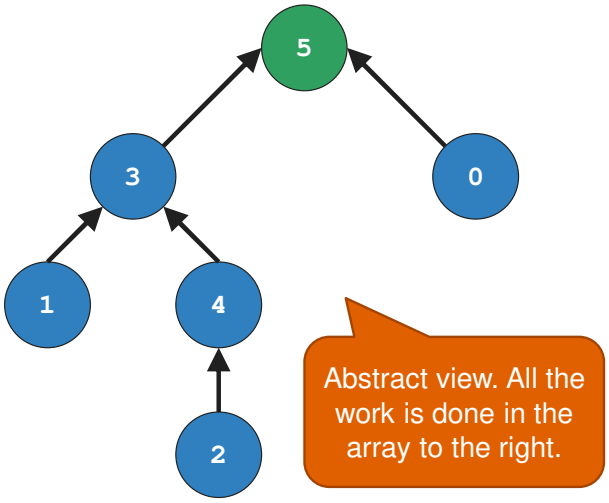

66

Fully Partitioned Set

- The table is initialized – when each item is singleton
- Notice that the parent is the same as the index

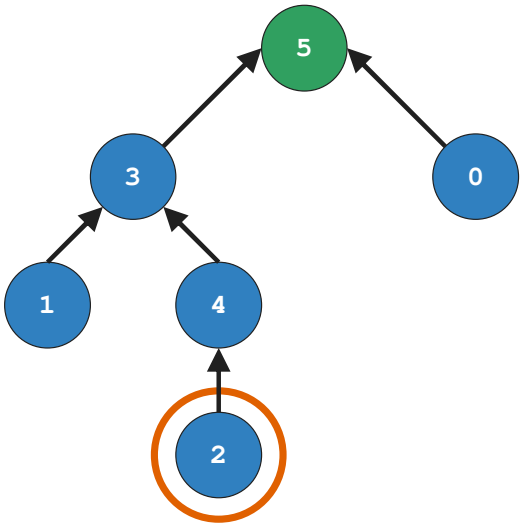
Index	Parent	Value
0	0	Frogger
1	1	Asteroids
2	2	Joust
3	3	Dig Dug
4	4	Galaga
5	5	Pac-man

Example Union-Find Array



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	4	Joust
3	5	Dig Dug
4	3	Galaga
5	5	Pac-man

Example: Recurse

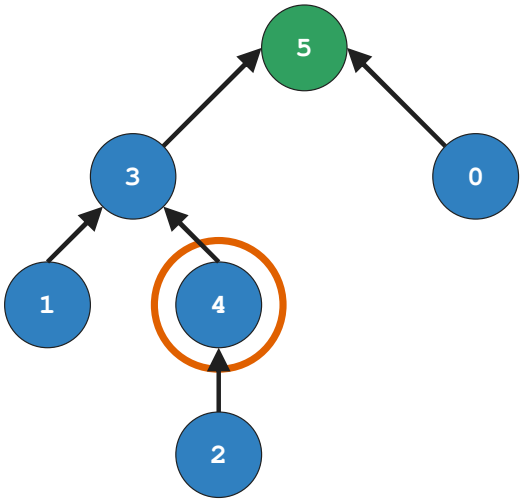


Index	Parent	Value
0	5	Frogger
1	3	
2	4	
3	5	Dig Dug
4	3	Galaga
5	5	Pac-man

Recurse using index 4

69

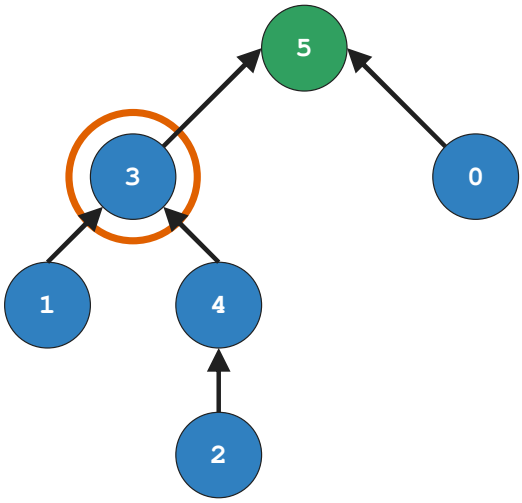
Example: Recurse



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	4	Joust
3	5	Dig Dug
4	3	Galaga
5	5	Pac-man

70

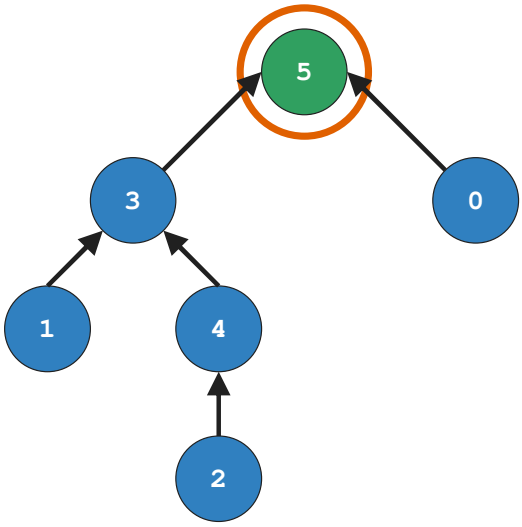
Example: Recurse



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	4	Joust
3	5	Dig Dug
4	3	Galaga
5	5	Pac-man

71

Example: Reached the Representative

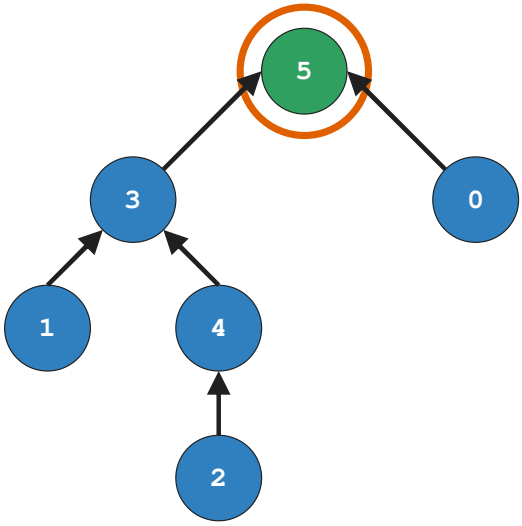


Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	4	Joust
3	5	Dig Dug
4	3	
5	5	

Representative is itself. Found!

72

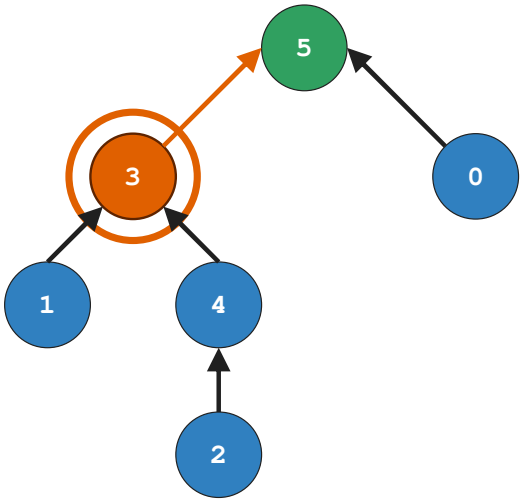
Example: Return from Recursion



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	4	Joust
3	5	Dig Dug
4	3	Galaga
5	5	Pac-man

73

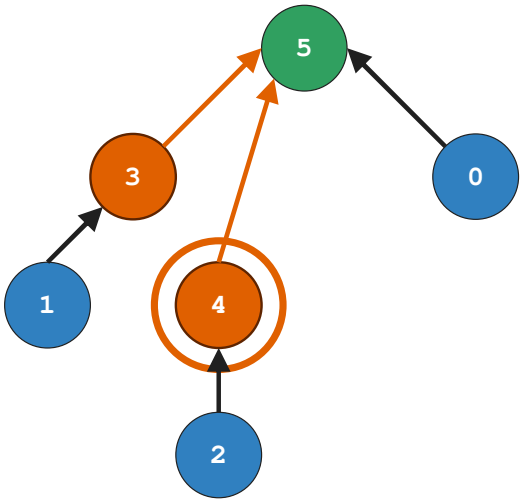
Example: Return from Recursion



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	4	Joust
3	5	Dig Dug
4	3	Galaga
5	5	Pac-man

74

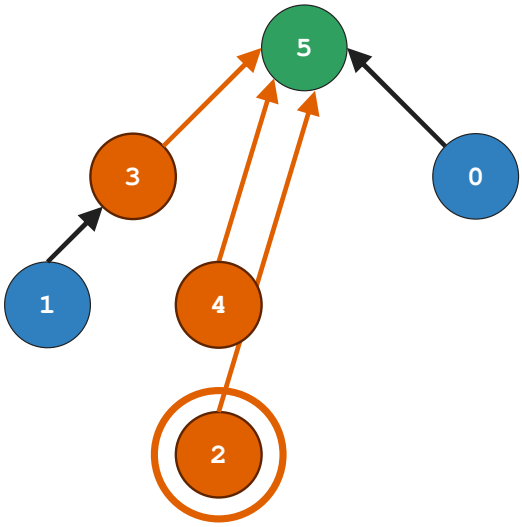
Example: Return from Recursion



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	4	Joust
3	5	Dig Dug
4	5	Galaga
5	5	Pac-man

75

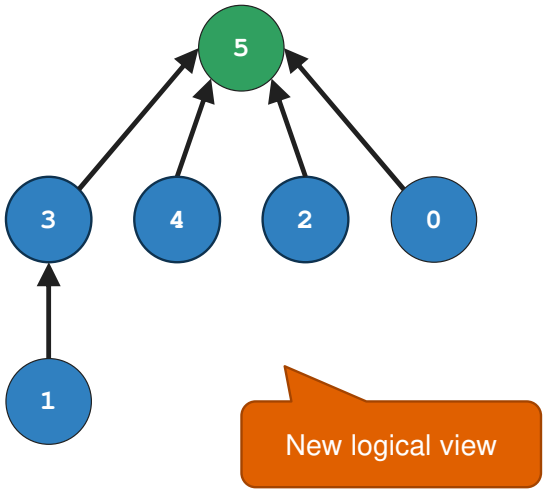
Example: Return from Recursion



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	5	Joust
3	5	Dig Dug
4	5	Galaga
5	5	Pac-man

76

Example: Resulting Table



Index	Parent	Value
0	5	Frogger
1	3	Asteroids
2	5	Joust
3	5	Dig Dug
4	5	Galaga
5	5	Pac-man